



# **Generative Adversarial Network (GAN)**

**Chao-Tsung Huang**

**National Tsing Hua University  
Department of Electrical Engineering**

# GAN learns data samplers



could also fail badly..

Ref: T. Karras et al, StyleGAN, ICLR 2018.

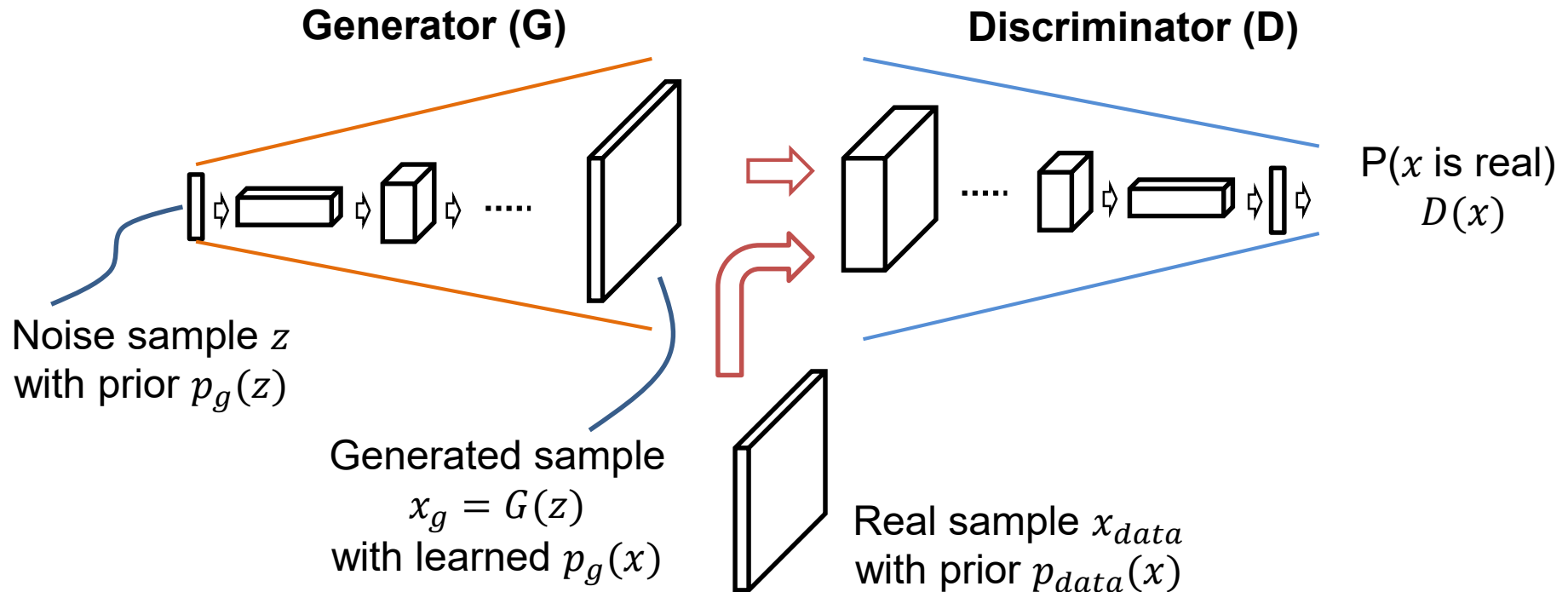


# Outline

- **GAN fundamentals**
- GAN architectures
- Loss functions for GAN training
- Regularization for GAN training



# Basic GAN architecture

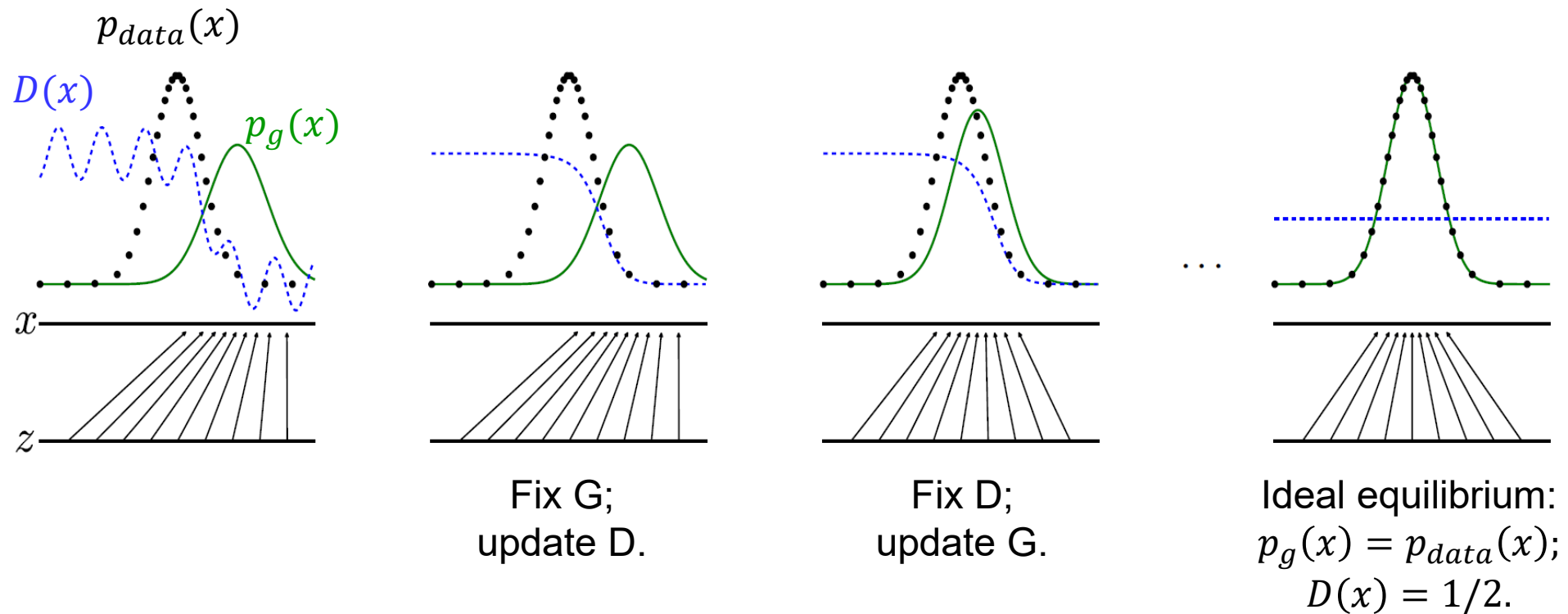


G tries to fool D, i.e.  $D(x_g) \rightarrow 1$ .

D tries to discriminate well between generated and real samples, i.e.  $D(x_g) \rightarrow 0, D(x_{data}) \rightarrow 1$ .



# Adversarial training process



toward unsupervised learning!

Ref: I. Goodfellow et al, NIPS 2014.



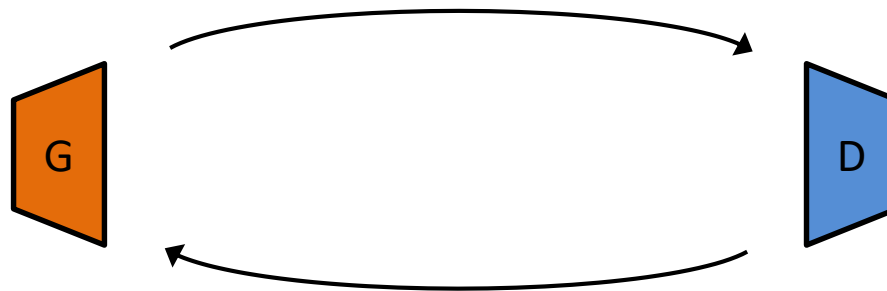
# Adversarial loss and its optimality

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Adversarial loss

$$\begin{aligned} V(G, D) &= \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) d\mathbf{x} + \int_{\mathbf{z}} p_z(\mathbf{z}) \log(1 - D(g(\mathbf{z}))) d\mathbf{z} \\ &= \int_{\mathbf{x}} p_{\text{data}}(\mathbf{x}) \log(D(\mathbf{x})) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) d\mathbf{x} \end{aligned}$$

$$D_G^* = \operatorname{argmax}_D V(D, G); D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$



$$C(G) = V(D_G^*, G) = -\ln 4 + 2 \cdot JSD(p_{\text{data}} \parallel p_g);$$

where Jensen-Shannon divergence  $JSD = KL\left(p_{\text{data}} \parallel \frac{p_{\text{data}} + p_g}{2}\right) + KL\left(p_g \parallel \frac{p_{\text{data}} + p_g}{2}\right)$ .

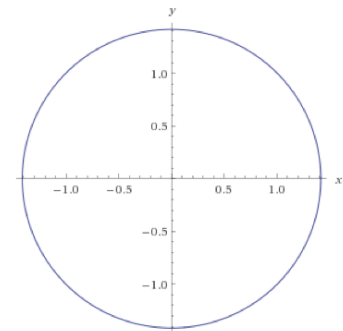
$$G^* = \operatorname{argmin}_G C(G) \Rightarrow p_g = p_{\text{data}}$$

Assumption: G has infinite capacity.

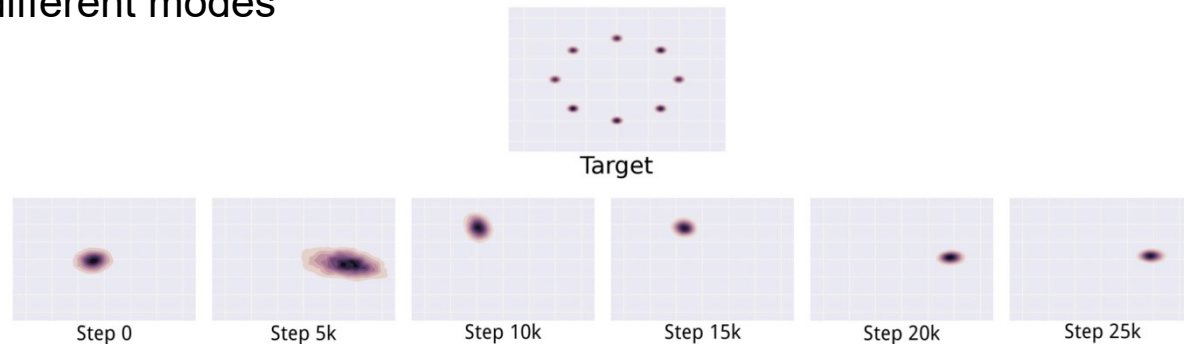


# Practical issues

- Heuristic loss for G
  - Adversarial loss will have zero gradient for G if D uses sigmoid output and  $D(x_g) \rightarrow 0$
  - Heuristic loss instead:  $J^G = -\mathbb{E}_z[\log D(G(z))]$
- Instability of simultaneous gradient descent
  - For some games, equilibrium is never approached.
- Mode collapse
  - G only generates a small family of images
    - D can discriminate many modes (and stop improving)
    - G cycles between different modes



$$V(x, y) = xy$$





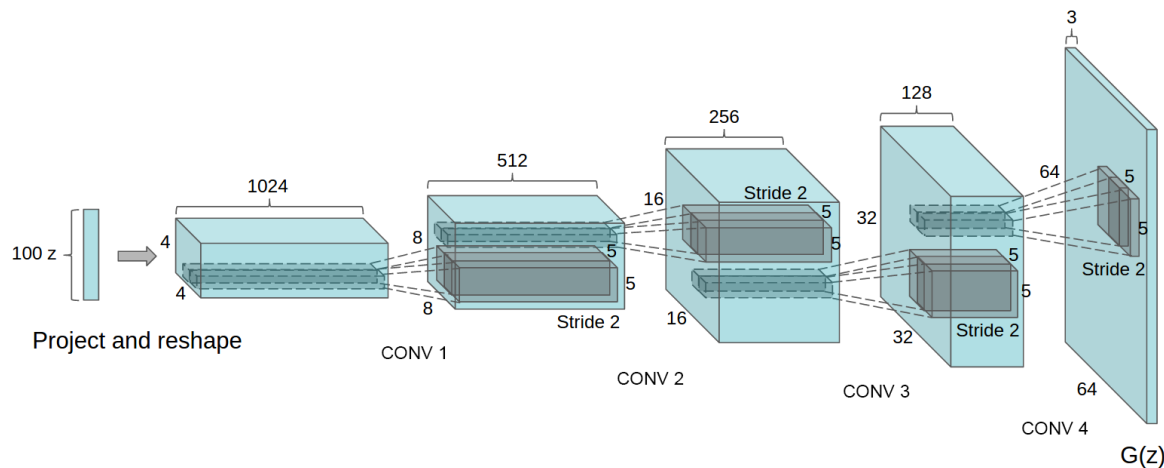
# Outline

- GAN fundamentals
- **GAN architectures**
- Loss functions for GAN training
- Regularization for GAN training



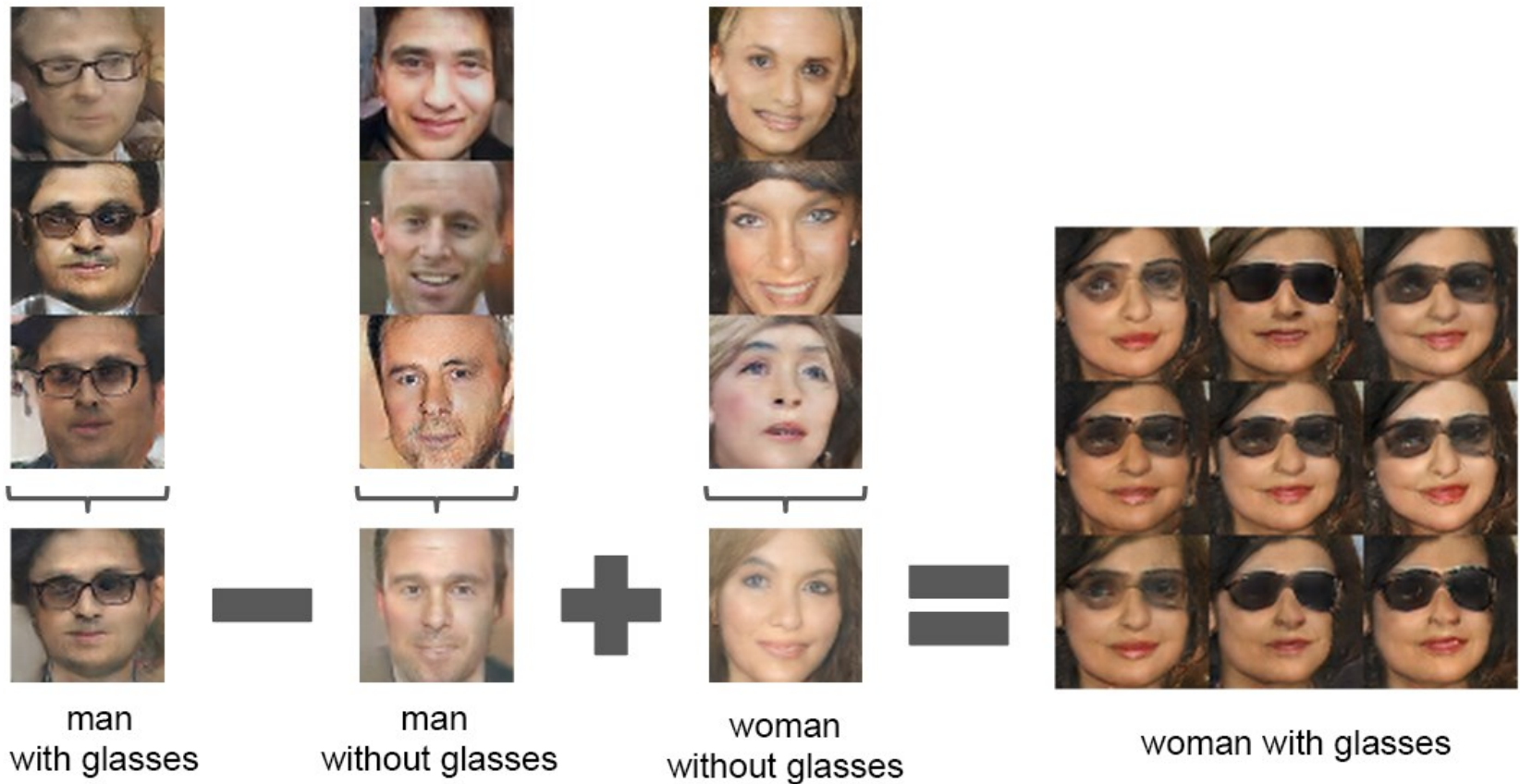
# Deep convolutional GAN (DCGAN)

- Architecture guidelines for stable training
  - Use strided convolution, not pooling, for D and upsampling convolution for G
  - Use BN in both G and D
  - Remove FC hidden layers
  - For G, use ReLU activation, except for the output, which uses tanh
  - For D, use leaky ReLU activation

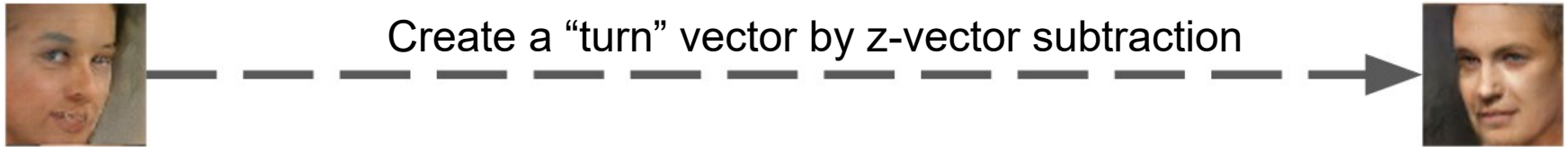


Ref: A. Radford et al, ICLR 2016.

# z-vector arithmetic for visual concepts

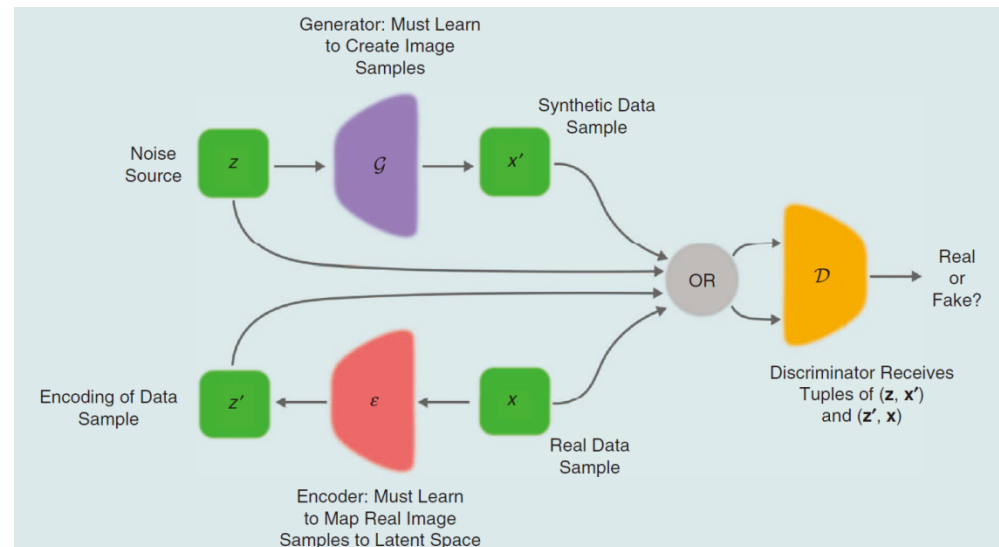
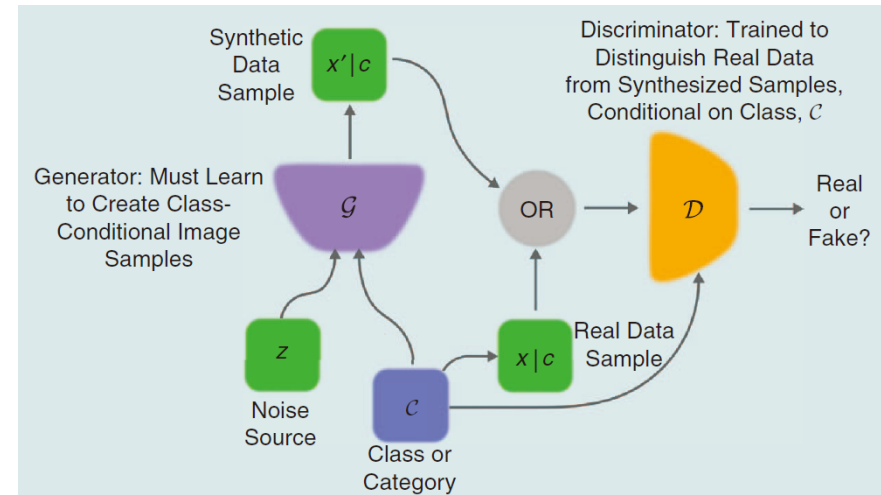


# z-vector arithmetic for visual concepts



# GAN variants on z-vector

- Conditional GAN
  - Create latent code  $c$ , e.g. category
- GAN with inference model
  - Map images into their codes  $z$



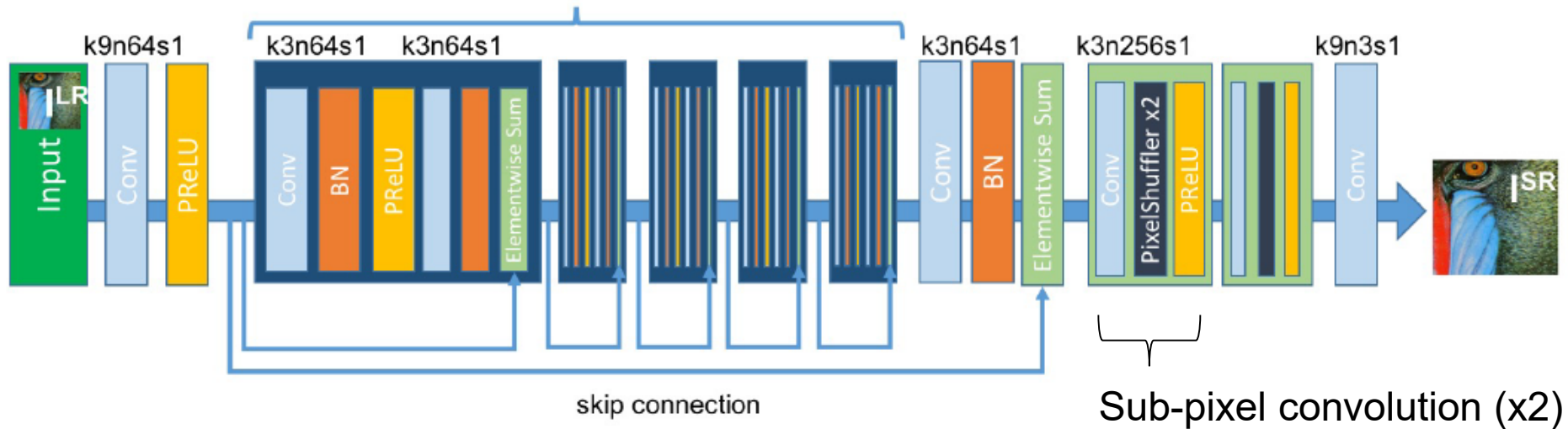
Ref: A. Creswell et al, IEEE SPM, 2018.

# SRGAN

$$l^{SR} = \underbrace{l_X^{SR}}_{\text{content loss}} + \underbrace{10^{-3} l_{Gen}^{SR}}_{\text{adversarial loss}}$$

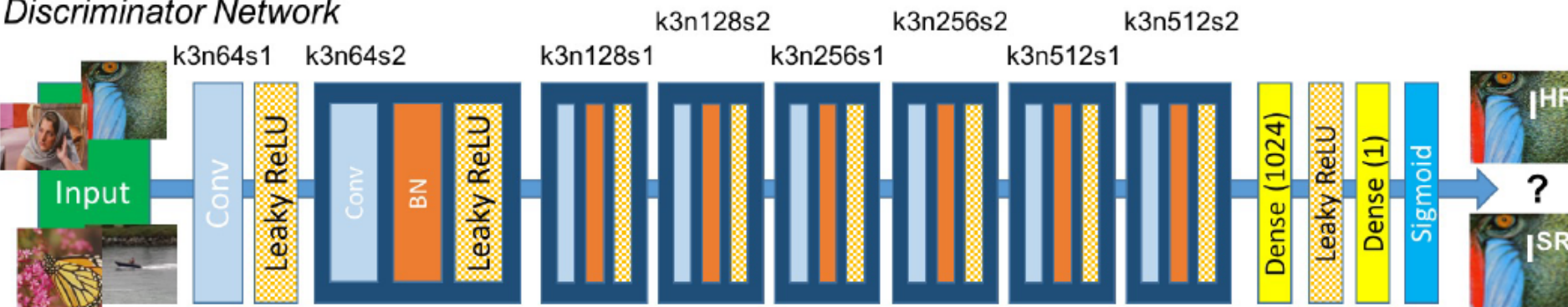
perceptual loss (for VGG based content losses)

## Generator Network

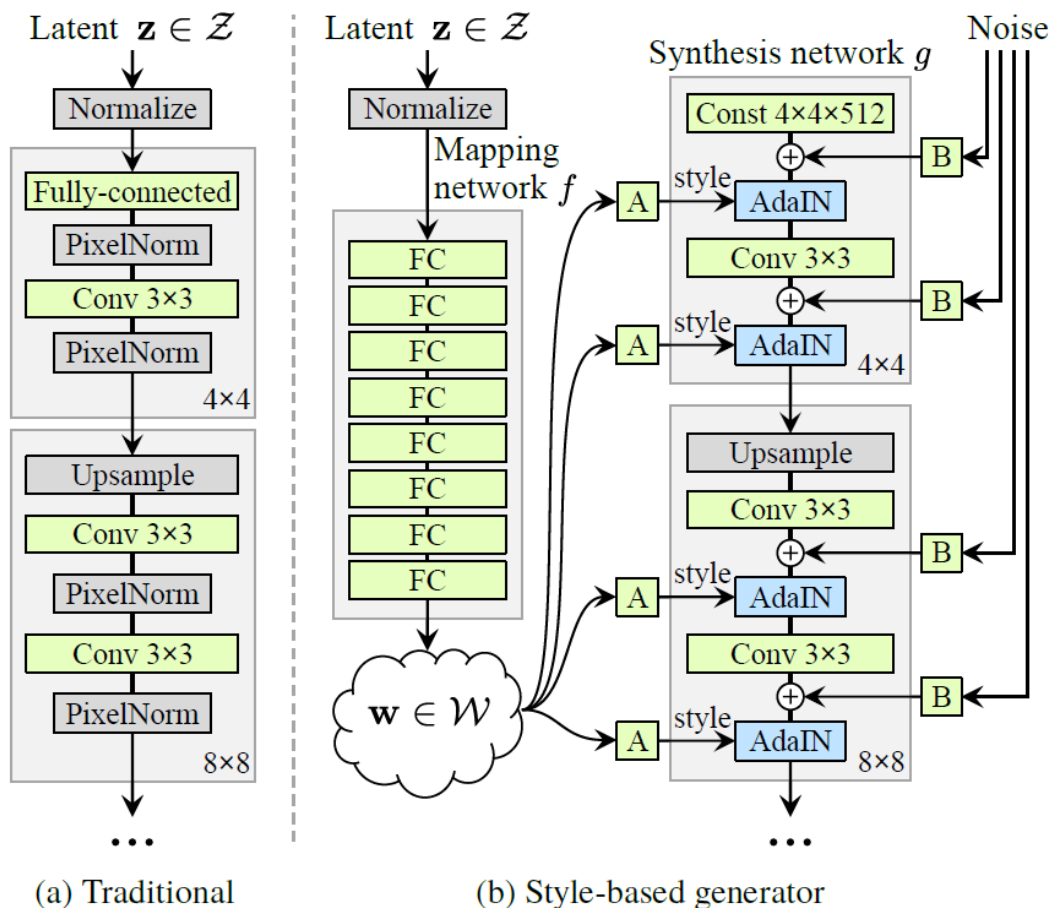


Sub-pixel convolution (x2)

## Discriminator Network



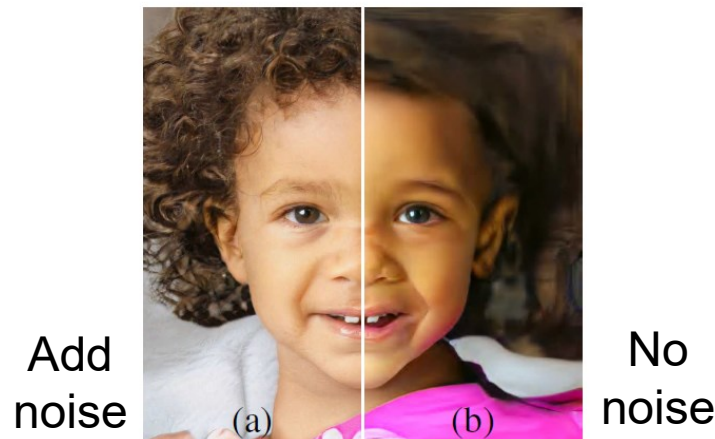
# StyleGAN



- AdaIN w/ layer-wise style  $y$ 
  - Styles calculated from a transformed intermediate latent code  $w$

$$\text{AdaIN}(\mathbf{x}_i, \mathbf{y}) = \mathbf{y}_{s,i} \frac{\mathbf{x}_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} + \mathbf{y}_{b,i}$$

- Add noise for stochastic local variations (more realistic)





# Outline

- GAN fundamentals
- GAN architectures
- **Loss functions for GAN training**
- Regularization for GAN training

Almost all about  $\nabla D \dots$



# How to define the distance between two distributions?

$$\delta(\mathbb{P}_r, \mathbb{P}_g) = \sup_{A \in \Sigma} |\mathbb{P}_r(A) - \mathbb{P}_g(A)|$$

Total variation

$$JS(\mathbb{P}_r, \mathbb{P}_g) = KL(\mathbb{P}_r \| \mathbb{P}_m) + KL(\mathbb{P}_g \| \mathbb{P}_m)$$

JS divergence  
(~adversarial loss)

$$KL(\mathbb{P}_r \| \mathbb{P}_g) = \int \log \left( \frac{P_r(x)}{P_g(x)} \right) P_r(x) d\mu(x)$$

KL divergence

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

Earth-Mover distance;  
Wasserstein-1  
(~Wasserstein loss)

Meaningful even when distributions are not overlapped!

Ref: M. Arjovsky et al, ICML 2017.



# Example: Learning parallel lines

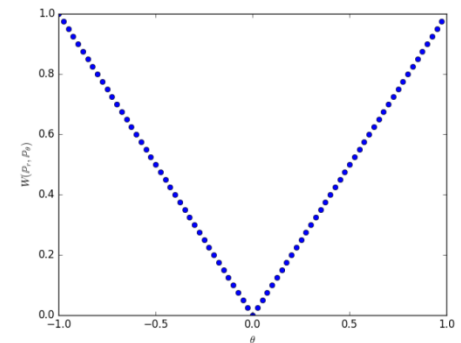
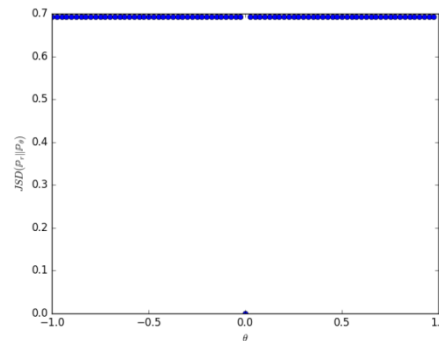
- Target: Uniform distribution on a vertical line between (0,0) and (0,1)
  - $\mathbb{P}_0$  is the distribution of (0, Z) where  $Z \sim U[0,1]$
- Generator: Uniform distribution between  $(\theta, 0)$  and  $(\theta, 1)$  where  $\theta$  is the parameter to learn
  - $g_\theta(z) = (\theta, z)$
- Only Wasserstein-1 provides meaningful loss to learn

$$JS(\mathbb{P}_0, \mathbb{P}_\theta) = \begin{cases} \log 2 & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

$$W(\mathbb{P}_0, \mathbb{P}_\theta) = |\theta|$$

$$\delta(\mathbb{P}_0, \mathbb{P}_\theta) = \begin{cases} 1 & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

$$KL(\mathbb{P}_\theta \| \mathbb{P}_0) = KL(\mathbb{P}_0 \| \mathbb{P}_\theta) = \begin{cases} +\infty & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$





# Wasserstein loss

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad \text{intractable}$$



Kantorovich-Rubinstein duality

$$W(\mathbb{P}_r, \mathbb{P}_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} [f(x)] - \mathbb{E}_{x \sim \mathbb{P}_\theta} [f(x)]$$

Lipschitz constant  $\|f\|_L = K$  of function  $f$ :  $|f(x_1) - f(x_2)| \leq K\|x_1 - x_2\|, \forall x_1, x_2$



WGAN

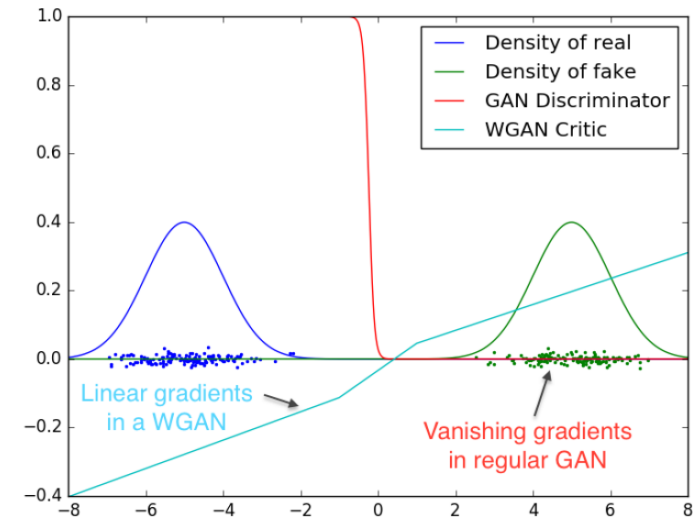
(1-Lipschitz constraint)

$$\min_G \max_f V(f, G) = \mathbb{E}_{x \sim p_{data}} [f(x)] - \mathbb{E}_{z \sim p_z} [f(G(z))]$$

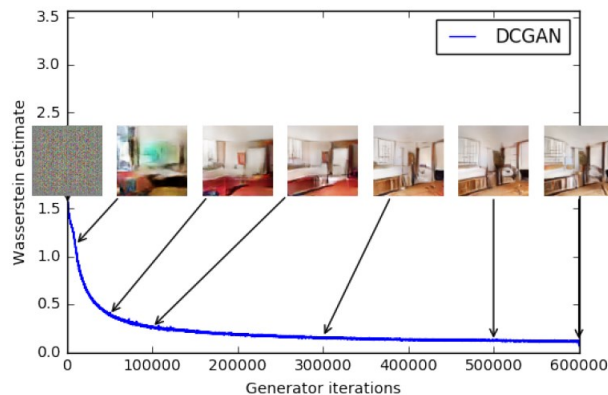
Critic  $f$  doesn't report probability of real/fake any more;  
Instead, higher  $f(x)$  means  $x$  is more likely real.

# WGAN over GAN

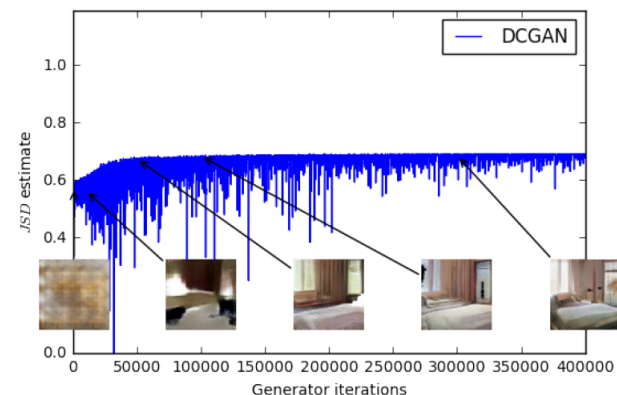
- Mode collapse is less likely to occur
  - WGAN loss always provides gradients for the critic and keeps it moving all the time
- Meaningful loss metric



WGAN loss decreases as G improves



JSD increases as G improves





# More loss functions to keep D moving

- Least-squares (LSGAN)

Ref: X. Mao et al, ICCV 2017.

- Equivalent to minimize Pearson  $\chi^2$  divergence if  $b=c+1=a+2$ 
  - $b=c=1$ ,  $a=0$  is used to make G learn real samples

$$\min_D V_{\text{LSGAN}}(D) = \frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [(D(\mathbf{x}) - b)^2] + \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [(D(G(\mathbf{z})) - a)^2]$$

$$\min_G V_{\text{LSGAN}}(G) = \frac{1}{2} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [(D(G(\mathbf{z})) - c)^2],$$

- Hinge loss

- D tries to push  $D(\mathbf{x})$  up to 1 and pull  $D(G(\mathbf{z}))$  down to -1
- G tries to push  $D(G(\mathbf{z}))$  as high as possible

$$V_D(\hat{G}, D) = \mathbb{E}_{\mathbf{x} \sim q_{\text{data}}(\mathbf{x})} [\min(0, -1 + D(\mathbf{x}))] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\min(0, -1 - D(\hat{G}(\mathbf{z})))]$$

$$V_G(G, \hat{D}) = - \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\hat{D}(G(\mathbf{z}))],$$



# Outline

- GAN fundamentals
- GAN architectures
- Loss functions for GAN training
- **Regularization for GAN training**
  - Implicit regularization
  - Explicit normalization

Try to confine the value range of weights to meet 1-Lipchitz.



# Weight clipping and gradient penalty for WGAN

- Try to enforce 1-Lipschitz constraint on D by
  - Weight clipping (WGAN)
    - Clamp the weights inside  $[-c, c]$ , e.g.  $c=0.01$
  - Gradient penalty (WGAN-GP)
    - Add gradient norm constraint as regularization

Ref: I. Gulrajani et al, NIPS 2017.

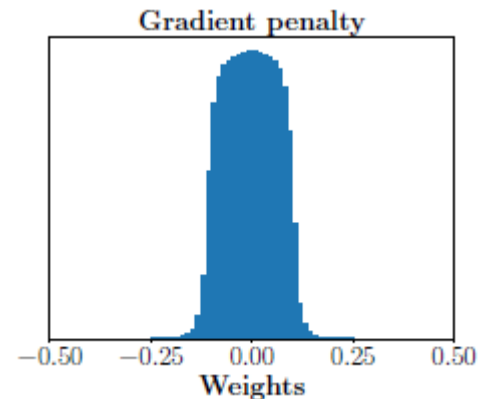
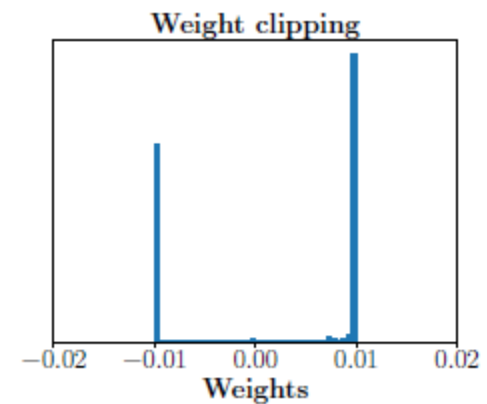
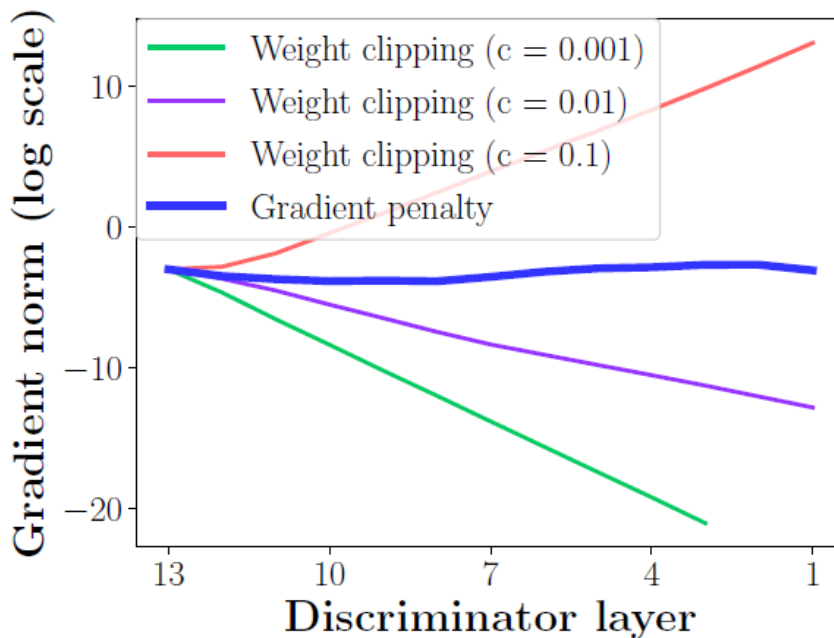
$$L = \underbrace{\mathbb{E}_{\tilde{x} \sim \mathbb{P}_g} [D(\tilde{x})] - \mathbb{E}_{x \sim \mathbb{P}_r} [D(x)]}_{\text{Original critic loss}} + \lambda \underbrace{\mathbb{E}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} [(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]}_{\text{Our gradient penalty}}.$$

- Reason: the optimal critic  $f$  has gradient norm 1 almost everywhere



# WGAN-GP over WGAN

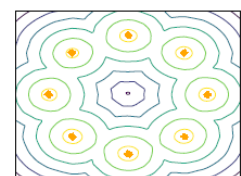
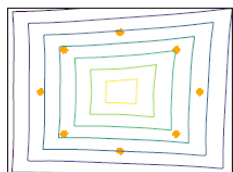
- WGAN may have exploded gradients and fail to capture high moments of data distribution



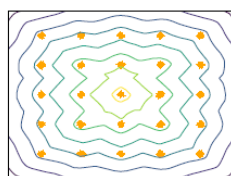
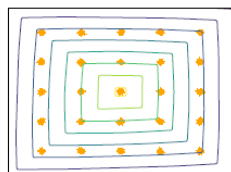
# WGAN-GP over WGAN

- WGAN may have exploded gradients and fail to capture high moments of data distribution

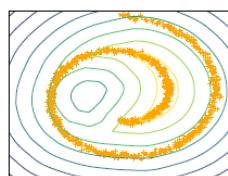
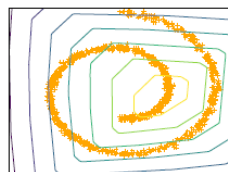
8 Gaussians



25 Gaussians



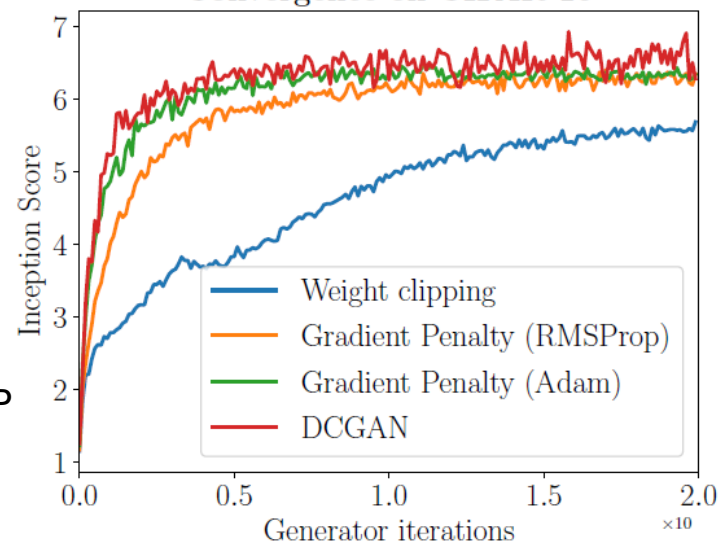
Swiss Roll



WGAN

WGAN-GP

Convergence on CIFAR-10



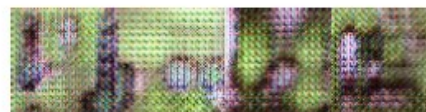
DCGAN

LSGAN

WGAN (clipping)

WGAN-GP (ours)

101-layer ResNet  $G$  and  $D$





# Other techniques for D

- Orthonormal regularization

- Add a new term to the loss  $\|W^T W - I\|_F^2$
- Con: enforcing orthonormal column vectors  $\Rightarrow$  singular values all approach to 1 (all features are equal)

- Weight normalization (explicit)

- Normalize L2-norm of each row vector of  $W$

$$\sigma_1(\bar{W}_{WN})^2 + \sigma_2(\bar{W}_{WN})^2 + \cdots + \sigma_T(\bar{W}_{WN})^2 = d_o, \text{ where } T = \min(d_i, d_o)$$

- Con: could result in rank-1  $W$  (one feature only)

$$\sigma_1(\bar{W}_{WN}) = \sqrt{d_o} \text{ and } \sigma_t(\bar{W}_{WN}) = 0$$

- Spectral normalization (explicit)

- Normalize  $W$  over the largest singular value, and Backprop goes through this normalization
  - Confine the value range of  $W$  while reserving feature diversity